Measured figures behind the code

Every number in `tsp/` that came from a measurement rather than a specification, with the conditions it was taken under. Comments in `tsp/` cite the headings here instead of carrying the derivations, so a constant can be re-derived without reading the module it lives in.

Firmware behaviour the manuals contradict or omit is in `UNDOCUMENTED.md`. This file is the app's own timings, geometry and arithmetic.

Instrument: DMM6500, firmware 1.7.17a, unless stated.

Reading a sample out of a reading buffer

`buf.readings` is a firmware proxy, so every index is an IPC round trip. Three independent measurements agree:

figure	how it was taken
21.73 us	the read component of a 20 000-sample decode step, timed per phase
22.75 us	derived from the 0.455 s a capture that `acq_copy_out` spends
22.82 us	timed directly, 20 000 reads in one loop

Independent of buffer style, so the cost is the round trip and not the payload: `STYLE_FULL` stores more per reading and reads within 0.07 % of `STYLE_STANDARD` (22.836 us against 22.820, three interleaved runs each).

Acquisition is the opposite -- 20 000 samples take 207.96 ms into a `STYLE_STANDARD` buffer and 221.65 ms into `STYLE_FULL`, +6.58 % at 8.3 sd. So payload size costs time where it is written and is invisible where it is read. `buffer.make` (1.8 ms) and `buf.clear()` (0.17 ms) are style-independent.

Streaming by real-time extraction is not possible

9600 baud at 10 samples/bit needs 96 000 samples a second: ~10 us a read with nothing left for decoding, ~5 us to leave half the budget. 21.73 us is 2.2x and 4.3x over those. Double buffering cannot help, the deficit being throughput rather than a gap between captures, so the app decodes one window at a time.

Decode phase timings

Timed per phase with `tools/bench_priming.py`, 9600 baud, W = 20 000:

phase	time	what it is
levels	0.958 s	the decimated pass, `ck_level_n` samples
reading	0.530 s	the contiguous lead-in read
edges	0.101 s	`sig_edges` + `sig_idle`
format	1.043 s	the format search, the most expensive per sample
window	0.685 s	one decode slice, x20

Every phase is proportional to W, so one per-sample cost bounds all five, which is what makes the latency bound real. Timing only `sig_edges` and `ua_run` over a window already in a Lua table understates the total threefold, because the buffer read is part of the cost.

ck_smp_us = 73

The format phase scales with FRAMES, and samples-per-bit varies 2:1. At W = 8653: 0.451 s at 9600 baud (8.33 sa/bit) against 0.634 s at 250 kBd (4.0 sa/bit). Two points give 32.5 us/sample + 1.63 ms/frame, so 52 us/sample at 8.33 sa/bit and 73 at 4.0. The worst is used: one always-safe constant beats two that must be evaluated before the format is known. Costs ~30 % window width at low rates -- throughput, not correctness.

ck_win_us = 48

One 20 000-sample decode step is 0.685 s, i.e. 34.25 us a sample, of which 21.73 is the buffer read and does not scale with frames. Only that one operating point was timed, so the worst case comes two ways: scaling the whole figure as the format phase scales from 8.33 to 4 sa/bit (x 73/52) gives 48, and holding the read fixed while doubling the frame-proportional remainder gives 47. The higher is used.

ck_slice_n = 1

Samples per press = W x n, and n = floor(budget / (W x ck_win_us)), so raising n by narrowing W cannot buy more than budget / ck_win_us samples per press: 9375 at 0.45 s against 6164 today. That 1.52x is the whole prize and taking it needs W <= 4687, i.e. n = 2. Three constraints then do not fit at 0.5 s -- n = 2, a W that stays a useful width (>= 5000, asserted in `test_serial.lua`), and the 0.9 margin. Any two hold: n = 2 with W = 5000 spends 0.480 s of the 0.5 s bound, inside it but on 4 % margin rather than 10 %.

Window geometry

Overlap

`ua_run` stops when a frame runs off the end of its array, so a window ending at W leaves at most one partial frame, whose start t0 satisfies t0 > W - framebits*T. The next window must therefore begin at or before W - framebits*T, giving `overlap >= framebits * T` as the floor.

Two further frames plus `maxback` buy two things. `sig_cross` walks back up to `maxback` samples to find a crossing and that walk is truncated at a window's leading edge, so the first edge or two can be mispositioned; with margin they sit in already-emitted territory and `tmin` discards them. The seam also falls mid-overlap rather than at its rim, so de-duplication never decides a near-tie.

At 8.68 samples/bit and 10-bit frames: ceil(260.4) + 64 = **325 samples, 1.6 %** of a 20 000-sample window.

Constant stride

The window loop advances by a constant stride rather than resuming where the last frame ended. That costs ~1.6 % in redundant decoding -- the overlap, once per window -- and buys a window count known before the run starts, which is what makes the progress indicator honest.

Level measurement

Why ck_level_n is 20 000

How many samples the level measurement decimates the recording down to:

count	bins	modal	threshold	sig_levels
982	61	0.585	3.263 V	accept -- threshold ABOVE the high level
1963	122	0.325	1.651 V	accept
3925	245	0.176	1.647 V	accept
5233	327	0.235	3.295 V	accept -- wrong again
7850	490	0.137	1.648 V	refuse
10466	512	0.146	1.645 V	refuse
31397	512	0.145	1.645 V	refuse

`sig_levels`' bin count is floor(n/binocc) capped at nbins, so it saturates at n = 8192 and every count past that measures the same thing. Below saturation the coarse bins sometimes pick the wrong pair of humps outright, so a small count is not merely noisier. `sdec.n` is past saturation and is frame mode's own population, so chunked mode accepts and refuses a line where frame mode does.

Why the contiguous fallback is spaced across the recording

Decimation can alias past a sparse burst: a short message in a long recording -- the traffic the streaming modes exist for -- reduces to a handful of samples once the step is 140, and can reduce to none. Reading only the first window covers **20 000 of the 3 413 334** a full 32 kB recording holds, 0.6 % of the burst positions, hence `ck_probes` contiguous probes spaced evenly. The decimated view is still tried first, so the drift property that motivated decimation is untouched.

Priming and the pre-trigger reserve

An armed record opens on silence

`LoopUntilEvent` keeps a pre-trigger reserve ahead of the trigger, and on a device that had not started talking when Capture was pressed, every reading of that reserve is the line the arm was waiting on: **71 865 readings of a 32 kB recording's 1 437 294**, which is 3.6 windows of 20 000. So the first window can be entirely the wait.

Measured at 9600 Bd locked: a device starting at +3.0 s gave **0 bytes** and "no frame format fits the capture", where the same recording of one starting at +0.3 s gave **21 933**.

ck_prime_tries = floor(leadmax / W) + 2

Plus two, never fewer than two, and the 8 kB mode is why: its reserve is 2 035 readings short of a window, so its first window is 90 % the wait. The window straddling the end of the reserve holds a sliver of signal which need not frame, so the try after the reserve is the first certainly on the device's own traffic. **5 tries for the 32 kB mode, 2 for the 8 kB mode.**

It is a bound because every try costs a window. Over the reserve a try is a read, an edge pass and a format search with nothing to search -- 0.63 s at W = 20 000 -- and on a window full of frames that do not fit it is the whole 1.67 s. The worst case is 5 tries at 32 kB: **8.4 s to refuse a recording that takes 1.67 s to refuse on one try.**

Why sig_idle must exclude the leading run on an armed record

The priming window of an armed recording is the reserve, so its leading run is the wire before the device started, not the line's idle level. Left unset, `sig_idle` takes that run as the longest in the window and the polarity prior comes out of it -- and because priming's own decode then frames at that polarity, the format it returns is inverted and the whole recording inherits it.

Measured, an 8 kB recording armed on silence with the generator switched on at +3 s: leadrun 17 968 against a 17 965-sample reserve, run0 17 968, run1 232, idle 0, invert true, **2110 of 5484 bytes bad**. The byte count is right, which is why a count-only assertion passes on it.

Mode caps

ck_maxbytes = 32768

One number for every baud rate, so the UI states one expectation. 32 kB is 400-500 lines of 64-column debug text. 32 rather than 64 kB because the worst case then needs **3.41 M samples**, under half the reading pool, so the defbuffers at their 100 000 defaults leave room -- and where they do not, `sdec.bufmax_for()` shortens the window instead of refusing.

Window reuse between priming and decode

`ck_prime_step`'s read phase fills `win[1..cnt]` from position 1, and nothing between it and `ck_decode` writes into the table, so re-reading the first window costs W proxy round trips for values already in hand -- **0.435 s at W = 20 000**. The reuse is conditional on four things, because a format search that found nothing in the reserve advances a window at a time and the table can hold samples from the middle of the record.

The USB file API

No file call raises

Measured with the key present. Every failure posts an event instead:

call	returns	posts
`file.write` to a closed handle	returns	2200
`file.flush` on a closed handle	returns	2211
`file.close` on a closed handle	returns	2202
`file.read` on a closed handle	nil	2201
`file.read` on a WRITE handle	nil	2201
`file.open` on a bad path	nil	2216
`fs.is_file` bad path or garbage	false	nothing
`file.write(fh, {})` -- a table	returns	1138, 'parameter 2, expected type string'

The table row is the strongest: even a Lua argument-type error becomes a posted event. Across every shape measured -- dead handle, wrong mode, bad path, wrong argument type -- a `pcall` round a file call returned true, so a verdict taken from one is a constant. **Presence is the only file failure this application can observe.**

The rule is measured, not universal: these are argument shapes on a healthy key. A key that is present and REFUSING -- full, or write-protected -- has never been exercised. `vendor/repro-04b-full-key.tsp` would settle it.

A key pulled from under a running app

Key out, handle still open:

call	pcall	posts
`file.write(stale, s)`	ok = TRUE, err = nil	2200, 'File write error'
`file.flush` / `file.close`	ok = true	nothing
`file.open(path, ...)`	returns nil	2224, 'USB flash device not present'
`file.usbdriveexists()`	returns 0	nothing

`file.usbdriveexists()` costs **237 us** with a key in, names no path and posts nothing, so it is askable per line, per capture and per panel refresh.

file.mkdir

Returns nothing, never raises, and posts **2208** on failure -- including for a name that already exists, so a speculative mkdir files one event per call. A bare name always fails; `file.mkdir('/usb1/SERDEC')` succeeds silently. The reference's own `file.mkdir("TestData5")` example is wrong. Opening a missing path posts **2205**, so probing for the directory is not free either.

The only silent way to tell is to read `/usb1`, which opens for read and yields its raw FAT directory table. With no key even `/usb1` posts 2205, hence `usbdriveexists()` first.

Why the directory is called SERDEC

A name of eight characters or fewer in upper case is a single contiguous 8.3 entry, found by one `string.find`. FAT splits a long name across fields of 5, 6 and 2 characters with record metadata between them, so no search for the whole name matches:

```
SERIALFILES = false     no plain entry exists for an 11-character name
utf16_full  = false     the UTF-16LE long name is NOT contiguous
utf16_5     = true      only its first five characters are
SERIAL~1    = true      a tilde alias exists, but its number is not predictable
```

The root listing is **65536 bytes**, measured, which is why a confirmed directory is cached.

fs.is_file against the open probe

`file.open(path, MODE_READ)` uses the failure as the answer and posts 2205 at severity ERROR, the one severity `showevents` still raises a modal box for -- so finding a free name would produce one on every Save. `fs.is_file()` answers silently: `eventlog.getcount()` stays 0 across an existing and a missing path.

LIN

The checksum's blind spot

Modulo-255 makes 0xFF congruent to 0x00, so swapping one for the other is invisible in every byte position: **0 of 256 detected across all 64 IDs and 4 positions.** With PID 0x11, data 0xFF and 0x00 both give checksum 0xEE. An agreeing checksum is therefore no proof that a byte the UART layer flagged arrived intact.

The checksum has **never been checked against a real frame** -- hand-computed cases only. `li_pid` is checked against the known headers: ID 0x3C -> PID 0x3C, 0x3D -> 0x7D, 0x00 -> 0x80, 0x01 -> 0xC1.

Break detection

`li_minbreak = 11` bit times. A break plus delimiter is at least 12 and a byte butted against the next is 10, so 11 separates them. Corroboration rather than an independent test: `li_break` also requires value 0x00 and a failed stop bit. What survives all three is a 0x00 whose stop bit flipped inside a stream with 2+ bit times of space, which splits one frame in two and fails both checksums.

Wiring

A 12 V single-wire bus against a 10 V best digitize range needs a 2:1 divider, 47 k + 47 k. Nothing below 20 k per leg, or a slave-only segment's pull-up reads recessive as dominant. A 24 V commercial-vehicle bus needs more than 2:1. `lin-divider-panel.png` is the drawing.

Distinguishing a divided bus from a transceiver TTL pin cannot work: 6 V against 5 V, and both decode anyway. So the panel note tests the recessive level only.

UART framing

Why only 7 and 8 data bits are searched by default

The hostile sweep run both ways changes 5 of its 105 UART cases, and every one changes the same way. Its LIN cases cannot change, LIN pinning the width at 8.

case	7/8 only	any width
60 spikes 5 V 8 wide	8N1, 2 err	9N1, 1 err -- bytes all +256
1 V drift over 2 cycles	8N1, 3 err	9N1, 0 err
1 V drift over 12 cycles	8O1, 5 err	9N1, 1 err
noise+spikes+ringing+drift	8N1, 15 err	9N1, 9 err
genuine 9N1 stream	8N1, 5 err	9N1, 0 err -- the one real gain

Two mechanisms, the second being the dangerous one. The search keeps the highest-scoring candidate, so every extra format is another draw at winning by luck: 2 widths x 3 parities x 2 polarities is 12 candidates, five widths is 30. And **a wider frame launders the evidence of damage** -- 9N1 has an extra data cell and its stop bit lands one place later, so it absorbs whatever broke the 8N1 stop check into a data bit instead of failing on it. That is why the error count FALLS as the answer gets worse, and every byte is 256 too large.

6 data bits does the same thing one cell down: 6 bits plus a stop plus two idle bits is exactly a 10-cell 8N1 frame, so on any capture with a 2-bit gap both score 12 and the loser's bytes are silently wrong.

Why 9 data bits is never searched, even biased

Biasing is inert exactly when it is needed. `ua_margin`'s proportional term is 0.6 * sc and applies only when sc > 0; on a damaged line the honest reading's own score is driven to zero by its correct error count -- 12 frames with 3 bad scores 9 - 3*3 = 0 -- so the term is 0, multiplying by `rarewidth` leaves 0, and the margin collapses to its floor of 2. A 9N1 rival scoring 12 clears that trivially.

Measured on an 8N1 line with a 2-bit gap and stop-bit damage on interior frames, "any width" selected:

capture	result
12 bytes, 3 stop bits corrupted	9N1, nbad = 0, **9 of 12 bytes wrong**, every one +256
20 bytes, 6 stop bits corrupted	9N1, nbad = 0, **14 of 20 bytes wrong**, every one +256

ERR reads 0 -- the one number an operator uses to decide whether to trust the dump -- and those bytes reach the USB byte log and the saved report. `ua_refine_width` cannot walk it back: its collapse requires the top bit set in every error-free frame, and since 9N1 reports no errors the damaged frames are all examined and the first with bit 9 = 0 breaks the loop. The same damage that lets 9N1 win disables the defence against it.

1.5 stop bits

capture	result
5N1.5 at 9600, auto	reports 19200, exactly double -- 8 frames, 4 bad
5N1.5 at 9600, baud forced to 9600	all six bytes exact, 0 framing errors

The frame layer handles it; `sig_bittime` does not. A 1.5-bit stop makes some pulse widths odd multiples of HALF a bit, so the greatest common bit period collapses to half the true one -- the "errs short" failure, but structural to the format rather than caused by a glitch, so noise tolerance cannot help.

Left unfixed deliberately: widening `sig_bittime` to resist a structural half-bit would put every common format at risk to rescue a doubly-rare one. 1.5 stop bits belongs to Baudot, which needs 5 data bits too, already outside the default search. The remedy is the Baud Rate field, and the failure is flagged rather than silent.

Baudot used a 7.5-unit character -- 1 start, 5 data, 1.5 stop -- and at a 22 ms bit that puts 45.45 Bd at the standard 60 words per minute. The mechanism needed settling time that was not an integer number of bit periods.

ratemargin = 0.04

Just inside the measured **5.26 %** cliff: 0.5/9.5 for 8N1, half a cell accumulated over the 9.5 bit times from the start edge to the stop sample. Advisory by design, warning at the point where the next percent could start corrupting bytes silently rather than waiting for corruption that by construction leaves no other trace.

qgate = 0.8, and why pulse-width consistency is needed at all

On a badly corrupted 9600 baud capture, 7N1 at 7200 baud frames **54 of 54 bytes with zero errors** while the truth frames 40 with 15 errors. A 9-cell frame at 13.87 samples spans 124.8 samples and a 10-cell frame plus a 2-bit gap at 10.417 spans 125, so the alias re-samples every frame at the wrong offsets and still lands its stop bit in the gap every time. No frame-error score can prefer the truth. Pulse-width consistency is decisive -- **0.855 against 0.497** -- because the real widths are multiples of the real bit time and of nothing else.

Why sdec.ratios is asymmetric

Integers to 10, reciprocals only to 1/6. `sig_bittime` fits the greatest common bit period of the observed pulses, so it errs SHORT: one narrow glitch pulls the fit to a sub-multiple, hence integer ratios well above 1. Erring LONG by a factor k needs every pulse in the capture to be an exact multiple of k bit times, and inside a 10-bit frame that stops being constructible past k = 6, where it would mean no run shorter than six bits.

ua_popcount's lookup table

The repeated-halving form is **11 % of every C call the app makes**, measured over 300 bench cells, because each iteration costs a `math.mod` and a `math.floor`. It is called once per CANDIDATE frame during the format and parity search, not once per decoded byte, so it runs thousands of times per capture. A 512-entry table over the 9-bit domain is ~8 kB against the ~500 kB a 32 k-sample capture already holds, and is built on first use so an app that never decodes a parity format never pays.

The loop is kept for an integer wider than the table, fused to one C call per iteration: `v * 0.5` has a fractional part exactly when v is odd, the same test as `mod(v, 2) == 1` for one floor instead of a floor and a mod. Not a general fallback -- a negative returns 0 and a non-integer counts fractional halvings -- but in-tree callers only ever pass 0..511.

Three-tap majority sampling

A single sample is one ADC conversion, so a noise excursion at that instant flips the bit; three taps spanning 40 % of the cell need two wrong together. Skipped below 6 samples/bit, where the outer taps would sit too close to the cell edges.

Two taps that agree settle a majority of three, so the third is not read: with the centre and lower tap on the same side of the threshold, `ones * 2 > votes` yields the same bit whether the upper tap is counted, counted the other way, or out of range -- all six cases check out. On a line clean enough to decode the pair agrees on essentially every slot, removing one `math.floor`, one bounds test and one array read per bit.

`ua_frame` repeats `ua_bit`'s sampling inline, expression for expression including the order of the floating-point operations, because `floor(x + 0.5)` at a cell boundary is decided by the last bit of x. A function call with its global look-ups costs about as much as the sampling it wraps.

Why the anchor search is bounded by `n`

The edge arrays span the whole capture and `n` need not: `ua_probe` scores a format over `ua_probe_n` samples from the first edge, and the edges past that are still in `ei`. Unbounded, the search returns an anchor beyond `n`, whose first frame reads off the end, comes back 'truncated' and breaks the walk before one frame is emitted -- so the probe reports that nothing frames while the format is right.

The failure needs the anchor reachable in the capture and not in the window, which a block payload produces exactly: 64 or 128 bytes of 0x55 put a one-bit mark run between every frame and never satisfy the 1.5-bit test, while the 0xAA block after them ends every frame with mark, mark and does. The anchor

lands in the 0xAA block, 129 bytes past a 48-byte probe window. Measured over the soak plan: **106 of 13 416 points**, every one v90 or v94 and no other vector, going to zero at every cap from 2000 samples up.

The bound is `n`, not "room for a whole frame". Requiring room for a frame also discards an in-window anchor that cannot complete one, and the fallback then frames from the window's start -- mid-byte on a payload with no 1.5-bit mark run of its own, so every frame carries a valid start and stop bit and a shifted value. Measured on 200 bytes of 0x12 with one 0xAA near the end: **201 frames, 201 marked good, every byte 0x22.**

A capture that began mid-byte

Measured on the instrument, 8E1 at 9600 into the front terminals: the frames ahead of the first idle come out at pitches of **11, 12 and 14.9 bit times instead of 13**, and every frame after it is exact. The cause is the anchor -- for the first edge the mark run is bounded only below, by the start of the capture, so two bit times of apparent mark is indistinguishable from a run of 1s mid-byte. An armed capture legitimately starts a few samples before a real start bit, so requiring a longer run cannot resolve it.

ua_edge_frames = 3

A capture of a continuously busy line starts and ends wherever the trigger and the buffer end put it, so the first and last frames are routinely sliced through the middle, and a gapless stream has no idle to anchor on so it also resynchronises a few frames in. Measured: without the exclusion, **every 9600-baud capture of a healthy line reports ERR 1 or 2** with the errors in the first row, colouring it red on a signal with nothing wrong. An error count never zero on good data is decoration, not a fault indicator.

ua_margin: why a plain '>' is wrong

Several formats are genuinely indistinguishable on the wire, and the shorter frame of such a pair always finds more frames in the same capture, so it always scores higher. 0x55 at 8N1 with a two-bit gap has a 12-bit period, and three 8-bit 6N1 frames tile two real frames exactly, so **6N1 decodes 9 error-free frames where 8N1 decodes 6**. Both are complete, consistent descriptions; no signal processing separates them.

The rule is therefore proportional -- a rival must win by >60 % -- which turns the search order into a commonality prior. Being scale-free it behaves the same on a 6-byte capture and a 600-byte one, and real evidence clears it easily: **a genuine 5N1 scores 8 against 8N1's 1.**

rarewidth = 8

At 8 a rare width must score nearly six times the incumbent and every genuine case still wins. **10 is the largest value that keeps every genuine 5N1 and 6N1 case decoding, and 12 breaks the tightest** -- 8 bytes of back-to-back 5N1, where 8N1 accidentally frames 4 of them and scores 1, so the rival needs to clear 0.6 * k. 8 leaves headroom rather than sitting on the wall; the 5N1/6N1-at-every-gap tests are the guard.

The bias multiplies the proportional term only, never the absolute floor. The floor stops noise deciding on a handful of frames, and quadrupling it made a rare width unwinnable on exactly the short captures where it is easiest to be sure: genuine back-to-back 5N1 over 8 bytes scores 8 against 8N1's 1, and a floor of 8 rejected it by one point.

polmargin = 4, and the inverted-ASCII trap

Three consecutive captures of the same 9600-baud line at 1 MS/s: the first decodes correctly, **the second flips to inverted and returns 18 frames with ZERO errors**, the third inherits it. At 104 samples/bit a 19 000-sample window holds only ~18 frames, and on that little evidence the inverted reading of ASCII text can out-score the correct one -- ASCII's always-zero top bit manufactures a periodic fake start edge nine bits after every real one. Self-consistent, confident, every byte wrong.

`sig_idle` sets `idle_weak` when the capture contained no idle, in which case the polarity prior is the signalling standard rather than an observation -- a single-supply line marks high, a ground-straddling one is RS-232 and marks low. A frame-count contest on one window should not overturn that on a 60 % margin, so the rival needs ~3.4x. 4 is the smallest value that blocks the 18-frame win while leaving every offline inverted-line test decoding; those captures contain idle, so the multiplier does not apply to them.

minsabit, and why the floor exists at all

Below the 4 samples/bit floor a clean synthetic waveform still decodes -- the harness recovers **13 of 13 bytes at 2.5 samples/bit** -- which is why the gate is explicit rather than left to emerge. Nothing real is that clean, and a decode that only works on a noiseless line is a trap.

plaustol = 0.02, on both ends

A genuine 250 kBd line measures **3.99998 samples per bit and 250 018 baud**, so refusing it for being two hundred-thousandths under its own floor would be a bug. The floor originally lacked the tolerance, with the mirror failure: a 50 Bd line whose fit measured 100.05 samples instead of 100.00 computed to **49.975 Bd and was refused**, while 51 Bd with identical jitter decoded byte-exact.

The median-pulse ceiling

Every pulse is a whole number of bit times, so every observed width is at least one bit and the median is too: a candidate longer than the median claims most pulses are shorter than a single bit. This rules out the long rescalings a damaged capture rewards -- at 1 V of drift the fit is right (**10.36 samples against a true 10.42**) but frames badly, and an 8x rescaling framing one byte takes over and reports 1200 baud.

ua_submultiple: the odd-multiple test

Halving is the dangerous direction: every pulse that is a multiple of T is also a multiple of T/2, so the q gate cannot separate them (**measured 0.99999999 for both**) while the halved reading finds twice the frames.

Odd multiples separate them. At the true bit time some pulses are an odd number of cells; at half it every pulse is even. Not the median -- all-0x00 is a nine-bit low run, so a median test rejected the truth. Even divisors only, and that is structural: an even divisor multiplies every k by an even number so nothing odd survives, while an odd one leaves k's parity as it was. Measured on the fox at 31250 Bd, 1 MS/s, odd/seen for 1/2..1/6: **0/358, 248/344, 0/333, 220/305, 0/305**. So 1/3 and 1/5 always pass whatever the payload, which is why `ua_minrun_absurd` exists.

It is a prior on real traffic, not a structural proof. A payload can have no odd run at all: repeated 0xE6 at 8N1 with a 2-bit gap gives runs of 2,2,2,6 -- **0 of 159 odd**. Measured on that case and on 0x33/0xCC/0x0F/0x66/0x00 at three gaps, the gate changes no answer for the worse and fixes two (0xE6 gap 1 refused -> correct, 0xCC gap 1 19200 -> 9600).

Both constants measured over **368 640 paired decodes**. A fraction rather than zero: jitter leaves 6-13 stray odd multiples per ~1000 pulses, and demanding exactly zero repairs only 398 of 708 cases. Odd fraction is 0.011-0.013 at a halved T against 0.587-0.604 at the true one.

Only bit runs may vouch, bounded by `maxmult`: a multiple beyond it is inter-byte idle carrying no timing information. A 42-bit loop-seam gap gives **k = 169**, whose parity is set by the capture phase rather than the format, and one such width was enough to silence the gate and let ratio 0.25 through.

ua_minrun_absurd = 2.5

Covers the odd divisors `ua_submultiple` structurally cannot. A ten-bit frame opens with a one-bit start bit, so hundreds of pulses hold single-bit runs: **1.00 bit times at the truth against 5.00 at the 1/5 candidate** that displaced it.

The bound must sit below 3, because a 3x misfit puts the shortest run at exactly 3.00 candidate bit times and 3x is the largest family on the bench. Measured on the LIN-break vector at 19200/57600/76800, 60 capture openings each: **2.0, 2.5 and 2.9 all report the true rate every time; 3.0 and 3.5 hand back 42, 60 and 60 wrong.** 2.5 is the middle of the working range.

What it costs: a payload whose shortest genuine run is 3 bit times or more cannot have a 1/3 or coarser rescaling admitted once the fit snaps. Repeated 0x00 at 8N1 with a two-bit gap is that shape, and reports a third of its true rate with this test and without it alike -- nothing lost there, but the exposure is real.

Over **1677 plan cells x 200 presses, 2.5 moves standard-rate misreports from 2604 to 882** and makes no cell worse. The 5th percentile rather than the minimum, because one 2-sample notch would silence this as one loop-seam width silenced the odd test -- and that notch is itself how a fifth-of-the-bit fit arises.

ua_fit_corrupt: med/Tfit separates two faults

The one number that separates "the line is faster than this instrument" from "impulse noise captured the fit", and structural rather than tuned: in async serial the median pulse is two bit times, because a ten-bit frame plus a couple of idle bits produces runs whose middle value is about two.

Measured med/Tfit is **2.0 for every healthy capture tried** -- 9600 clean, 250 kBd, 285 kBd, 921 kBd, and all the drift cases -- and **3.0, 11 and 13 for the three spike-corrupted fits**.

So when the fit is implausible AND consistent with its own widths, the line really is out of range and rescaling would manufacture a plausible-looking wrong answer. When it is grossly inconsistent, the fit is what is wrong and rescaling is right.

Why ua_cands adds median-derived candidates

The least-squares fit has one failure mode the ratio ladder cannot cover. **Forty 2-sample spikes on a 9600 baud line** produce forty narrow widths that corroborate each other, so seed corroboration passes them and the fit lands at **0.95 samples against a true bit time of 10.42** -- a ratio of 11, which is not on the ladder and never will be for every such case. Coverage scoring cannot reject it either, because a shorter trial period reclassifies the contradicting long widths as idle and drops them from its own denominator.

The median cannot be moved that way: spikes are a minority of the widths however many there are. Real widths are 1..maxmult bit times, so med/k for small k brackets the truth from a direction impulse noise cannot corrupt, and **med/1 alone recovers all three spike cases the ladder missed**.

Why the rescaling search is gated

Halving the bit time roughly doubles the frames found, so any score counting frames, bits or coverage is biased toward a smaller bit time: **on a repeated 0x55 square wave the true bit time frames 6 bytes and a quarter of it frames 30, all error-free.** Every rescaling of an alternating pattern is self-consistent, so no scoring function can prefer the truth.

`sig_bittime`'s two blind spots on synthesised captures: the widths can share a common factor (0x00 at 8N1 with a two-bit gap yields only 9- and 3-bit runs, best-fitting **three** bit times), and a sub-bit glitch can score higher than the truth because a shorter trial period draws fewer long widths into its coverage measure (a 2-sample notch produced a fit at **one fifth** of the real bit time, **q = 0.984 against the truth's 0.962**).

So the width fit is trusted whenever it snaps to a standard rate and frames something; otherwise the snapped rescalings are searched, real devices using standard rates being the strongest prior available.

The same waveform at two rates

A common factor in the pulse widths does not reliably produce a non-standard rate, because half of a standard rate is usually also standard. **Eight 0x00 frames at 9600 7N1 with a one-bit gap fit 4800 exactly, and read as four 0x08 bytes at 4800 8N1 with zero errors** -- and that is not an error, because the two encodings are bit-for-bit the same waveform, verified sample by sample. Which device is on the wire cannot be recovered from it.

Both readings are legitimate and the commonality prior even favours the 8N1 one, so the operator is told: a rescaling that also snaps and explains materially more of the capture is reported in `baud_note`. Perfectly periodic traffic creates this; irregular inter-byte gaps break the coincidence, so it is largely a synthetic-stream phenomenon.

ua_probe_n = 4000

The search is the app's worst latency offender: **4.376 s to auto-detect 38400 baud** on the instrument, against a 500 ms press-latency budget. The cost is multiplicative -- 3 widths x 3 parities x 2 polarities is eighteen full framing passes per candidate bit time, and `ratios` offers thirteen candidates before the integer sub-multiples -- so a probe pass runs upward of two hundred times.

A probe's output is a relative score used only to rank candidates, and ranking is settled long before the last byte. The final decode is unaffected; only the ranking is truncated.

4000 is measured, not chosen. Sweeping the cap against the 148-case hostile suite, edge-anchored:

cap	result
none, 16000, 12000, 8000, 6000, 4000, 2000, 1500	127 ok, 21 degraded, 0 WRONG, 0 raised
1000	126 ok, 22 degraded, 0 WRONG, 0 raised
500	125 ok, 22 degraded, **1 WRONG**, 0 raised

Discrimination is intact down to 1500, starts costing at 1000, and produces a wrong answer at 500. 4000 keeps 2.7x margin over the degradation floor while cutting the search's work 5x against a 20 000-sample capture. On the host the whole search costs **3.3x more uncapped than at 4000**, which puts the instrument's 4.376 s at about 1.1 s. It is ~48 bytes at 8.33 samples/bit.

Re-run that sweep (`SDEC_PROBE_N=N lua tools/stress_serial.lua`) before changing it. The failure mode of too small a value is a quietly worse answer, not a crash, and only the suite makes it visible.

The cap does not decide whether a block payload decodes, though it can look as though it does. v90 and v94 lost 106 of 13 416 soak-plan points to 'no frame fits' and the count fell monotonically as the cap rose -- that was the frame anchor being chosen outside the window the probe scores, not the window being too small. See 'Why the anchor search is bounded by `n`'.

ua_best's split

`ua_best_probe` scores one candidate and is the expensive unit, **~76 ms measured at 38400 baud**. A candidate probe is the loop's atom -- 18 framing passes with no state carried between candidates except the best-so-far -- so the checkpoint is four values and a slice boundary cannot land anywhere awkward.